Brainfuck Interpretation and Optimization

Max Krummenacher

Contents

1. Brainfuck 3
   1.1. Comments 3
   1.2. Examples 3
2. Intermediate representation 5
3. Canonicalization of Basic Blocks 6
   3.1. Collecting `MOV` 6
   3.2. Sorting Operations 6
   3.3. Combining and Eliminating Operations 6
4. Loop elimination 7
5. Underflow and Overflow Handling 8
   5.1. Limmitations 8
6. Edge Case Behavior 8
7. Translation to Aarch64 Assembly 8
   7.1. Prologue 8
   7.2. Epilogue 9
   7.3. (`MOV`, n) 9
   7.4. (`ADD`, n, o) 9
   7.5. (`LENTR`, a) 9
   7.6. (`LEXIT`, a) 10
   7.7. (`PCH`, n) 10
   7.8. (`GCH`, n) 10

1. Brainfuck

Brainfuck is an minimalist language operating on a tape of zero initialised unsigned bytes through a pointer to that tape. The tape is bounded on the left and unbounded to the right, although implementations usually use an upperbound for practical purposes.

The operations of brainfuck are shown in Table 1.

Instruction	Effect	C equivalent
`+`	increase the value pointed at by one	`(*t_ptr)++;`
`-`	decrease the value pointed at by one	`(*t_ptr)--;`
`>`	increase the tape pointer by one	`t_ptr++;`
`<`	decrease the tape pointer by one	`t_ptr--;`
`[`	if the value pointed at is zero goto the instruction after the matching `]`	`while (*t_ptr != 0) {`
`]`	if the value pointed at is non zero goto the instruction after the matching `[`	`}`
`.`	output the byte at the tape pointer	`putchar(*t_ptr);`
`,`	get one byte of input and write it to the tape	`*t_ptr = getchar();`

Table 1: Brainfuck Instructions

1.1. Comments

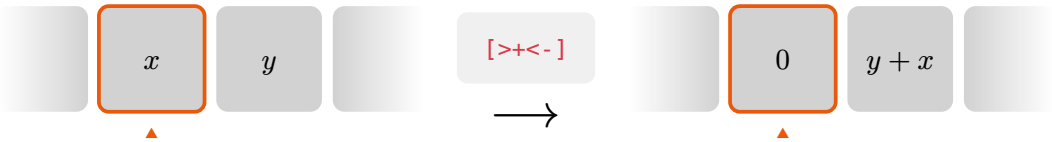
Any character which does not match any of the described operations is considered a comment. Additionally, if a program starts with `[` anything until the matching `]` can be considered a comment. This second type of comment can use any character, including operations, with the only limmitation that `[` and `]` need to be matched. Since the tape is zero initialized any code written here is skipped.

1.2. Examples



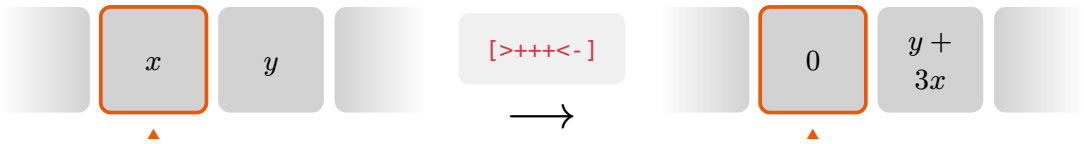
Listing 1: Clear loop

Listing 1 shows the transformation to clear the value at the current tape pointer. This loop decrements the value at the tape pointer by one until zero is reached. Note that this loop runs exactly x times.



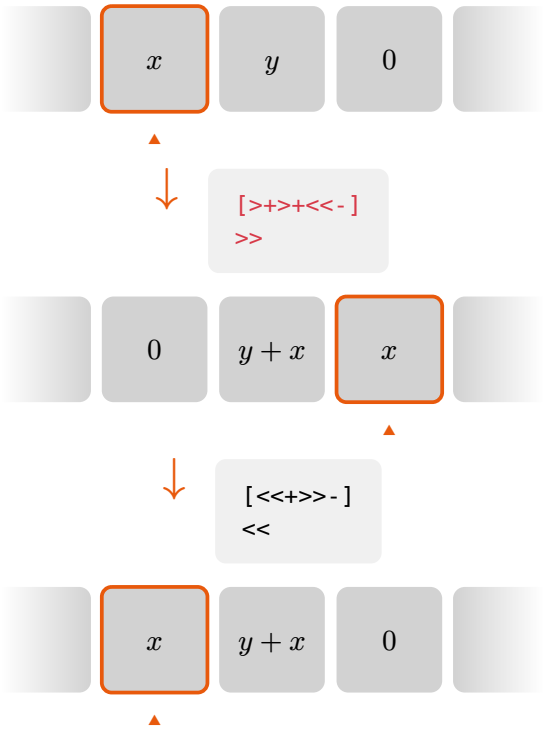
Listing 2: Addition

The loop shown in Listing 2 adds the value at the tape pointer to the next value on the tape pointer. This works because the loop above runs exactly x times and increases the value by one each time.



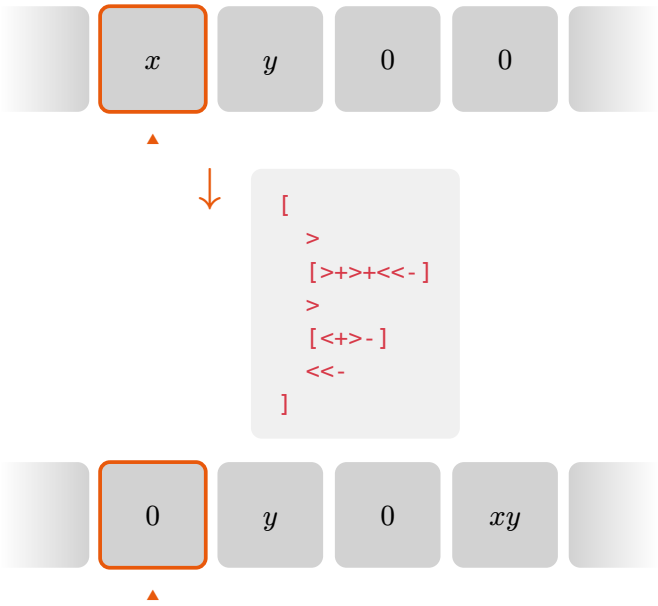
Listing 3: Multiplication by Constant

The loop shown in Listing 3 works in a simmilar fashion to the example in Listing 2, with the difference that the increment operation is run three times which multiplies the value at offset 0 by three. This can be generalized to a multiplication by an arbitrary constant.



Listing 4: Addition Preserving Original value

The loop shown in Listing 2 adds the value at the tape pointer to the next value on the tape pointer.



Listing 5: Dynamic Multiplication

2. Intermediate representation

The brainfuck code is first translated into an intermediate representation (IR). The instructions can operate on an immediate offset to the tape pointer. These offsets are indicated with `^`.

The operations of the IR are shown in Table 2.

Operation	Description	C equivalent
`ADD ^off imm`	add `imm` to the value at tape pointer + `off`	`*(t_ptr + off) += imm;`
`MOV imm`	move the tape pointer by `imm`	`t_ptr += imm;`
`LENTR`	jump forward to the instruction after the matching `LEXIT` if the value at the tape pointer is zero	`while (*t_ptr != 0) {`
`LEXIT`	jump backward to the instruction after the matching `LENTR` if the value at the tape pointer is non-zero	`}`
`CENTR ^off`	jump forward to the instruction after the matching `CEXIT` if the value at the tape pointer + `off` is zero	`if (*(t_ptr + off) != 0) {`
`CEXIT`	do nothing, this is just the target for the `CENTR` instruction	`}`
`PCH ^off`	output the value at tape pointer + `off`	`putchar(*(t_ptr + off));`
`GCH ^off`	get a byte of input and write it to tape pointer + `off`	`*(t_ptr + off) = getchar();`
`MAD ^dst ^src imm`	multiply the value at tape pointer + `src` by `imm` and add the result to the value at tape pointer + `dst`	`*(t_ptr + dst) += *(t_ptr + src) * imm;`
`MUL ^dst ^src1 ^src2 imm`	multiply the value at tape pointer + `src1` by the value at tape pointer +`src2` and by `imm` and add the result to the value at tape pointer + `dst`	`*(t_ptr + dst) += *(t_ptr + src1) * *(t_ptr + src2) * imm;`
`NEG ^off`	negate the value at tape pointer + `off`	`*(t_ptr + off) = -*(t_ptr + off);`
`CLR ^off`	set the value at tape pointer + `off` to zero	`*(t_ptr + off) = 0;`
`NOP`	do nothing	-

Table 2: IR-Instructions

BF-Instruction	IR-Instruction
+	ADD ^0 1
-	ADD ^0 255
>	MOV 1
<	MOV -1
[	LENTR
]	LEXIT
.	PCH ^0
,	GCH ^0

Table 3: Translation to IR

3. Canonicalization of Basic Blocks

A basic block is a segment of IR which does not contain any of `LENTR`, `LEXIT`, `CENTR`, `CEXIT`. Canonicalization of such blocks happends in three stages.

- collecting the `MOV`-operations into one.
- sorting the operations
- eliminating and combining operations.

3.1. Collecting `MOV`

The `MOV` operations are collected and put at the end of a basic block and the offsets to the arithmetic and io operations are updated accordingly.

```
ADD ^0 1
MOV  1
ADD ^0 1
CLR ^1
```

```
ADD ^0 1
ADD ^1 1
CLR ^2
MOV  1
```

Listing 6:

3.2. Sorting Operations

To ensure correct sorting of the instructions bubble sort with an additional commutativity test is used.

The special case of the commutativity of `MOV` can be ignored, since the `MOV` instructions were already moved to the end of the basic block in the last step of the canonicalization. The IO-operations `PCH` and `GCH` never commute.

The commutativity of the other instructions can be determined from their foot print.

3.3. Combining and Eliminating Operations

After sorting, operations on the same offset are adjacent unless a non-commuting operation separates them. Adjacent operations `a; b` with the same destination offset are merged according to Table 4. A `NOP` produced by merging is removed in the same pass.

First	Second	Result
`ADD ^o x`	`ADD ^o y`	`ADD ^o x+y` (`NOP` if `x+y` is 0)
`ADD ^o x`	`CLR ^o` / `GCH ^o`	second operation only
`NEG ^o`	`NEG ^o`	`NOP`
`NEG ^o`	`CLR ^o` / `GCH ^o`	second operation only
`CLR ^o`	`CLR ^o` / `NEG ^o`	`CLR ^o`
`CLR ^o`	`GCH ^o`	`GCH ^o`
`MAD ^d ^s x`	`MAD ^d ^s y`	`MAD ^d ^s x+y`

Table 4: Combination rules

`PCH` is never merged, since each output is observable. `GCH` is never removed, since each input byte must be consumed.

4. Loop elimination

Conditional exectution

```
LENTR
ADD    ^0 -1
ADD    ^o  n
LEXIT
```

```
CENTR  ^0
MAD    ^o ^0 n
CLR    ^0
CEXIT
```

for o$\neq 0$

the rest is stale

```
LENTR
ADD ^0  1
ADD ^o  n
LEXIT
```

```
NEG ^o
MAD ^o ^0 n
CLR ^0
```

```
LENTR
ADD ^0 -1
MAD ^d ^s n
LEXIT
```

if $0 \neq d \neq s \neq 0$

$$x'_d = x_d + nx_s + ... + nx_s = x_d + x_0 + x_0 n x_s$$

```
MUL ^d ^s ^0 n
CLR ^0
```

```
LENTR
ADD ^0 -1
CLR ^o
LEXIT
```

stale until here

5. Underflow and Overflow Handling

Underflows and overflows are handled by the operating system. The tape region is a multiple of the page size of the system, which allows me to allocate a page with no permission below and above the valid region. Out of bounds access is then handeled by catching the segfault and is determined to be an underflow or overflow by which of the guard regions is accessed.

5.1. Limmitations

If the memory access by brainfuck is outside the bounds by more than a region an unknown error is returned, as I cannot be sure if the access was meant to access the tape.

The validity of a tape address is only checked on access to a value. The tape pointer is allowed to be invalid if it is never accessed. The programm `<> +` does not report an underflow even though after `<` it points outside the tape region.

6. Edge Case Behavior

memory access of

```
[-<+>]
```

7. Translation to Aarch64 Assembly

The assembly for the one brainfuck file is translated to a single function: `void bf_assembled(void *tape_base, int *putchar(int), int *getchar(void));`

One register holds the tape pointer (`xtape`) throughout the whole program.

`xtape` is chosen to be x19, since it is a callee saved register.

To call `getchar` and `putchar` the function pointers are passed to the function. Simmilar to `xtape` callee saved registers (`xgetc` and `xputc`) are used for them.

7.1. Prologue

The following prologue is the header to the function body. It sets up the registers and the stack for the execution of the program.

```
; allocate frame, save {fp, lr} at [sp, 0]
stp fp, lr, [sp, -48]!

; point fp to {fp, lr} record
mov fp, sp
```

```
; preserve callee preserved registers
stp xtape, xunused, [sp, 16]
stp xputc, xgetc, [sp, 32]

; move arguments into correct registers
mov xtape, x0
mov xputc, x1
mov xgetc, x2
```

7.2. Epilogue

The following epilogue is the footer to the function body. It restores the registers and deallocates the stack.

```
; restore callee saved registers
ldp xtape, xunused, [sp, 16]
ldp xputc, xgetc, [sp, 32]

; restore {fp, lr} and deallocate
ldp fp, lr, [sp], 48
ret
```

the following sections are stale

7.3. (MOV, n)

If n > 0

```
add xtape, xtape, n
```

If n < 0

```
sub xtape, xtape, -n
```

7.4. (ADD, n, o)

If n != 0

```
ldurb w0, [xtape, o]
add w1, w0, (n % 256)
sturb w1, [xtape, o]
```

7.5. (LENTR, a)

```
ldrb w0, [xtape]
cbz w0, offset(a)
```

7.6. (LEXIT, a)

```
    ldrb w0, [xtape]
    cbnz w0, offset(a)
```

7.7. (PCH, n)

```
    ldrb w0, [xtape]
    blr xputc
```

repeated n times

7.8. (GCH, n)

```
    blr xgetc
    strb w0, [xtape]
```

repeated n times